

Modelos de Redes Neurais: Recorrentes, Competitivas

Davi Duarte de Paula
davi.duarte@unesp.br

Redes Neurais Recorrentes

Introdução à RNN

- Redes neurais recorrentes (RNNs) foram projetadas para **dados sequenciais**:
 - Textos (sentenças, linguagem natural)
 - Séries temporais (financeiras, meteorológicas etc.)
 - Sequências biológicas (DNA, proteínas)
- Entrada é uma sequência:
 $x_1, x_2, \dots, x_n$
cada x_t é um vetor d-dimensional no instante t.
-

Representação dos Dados

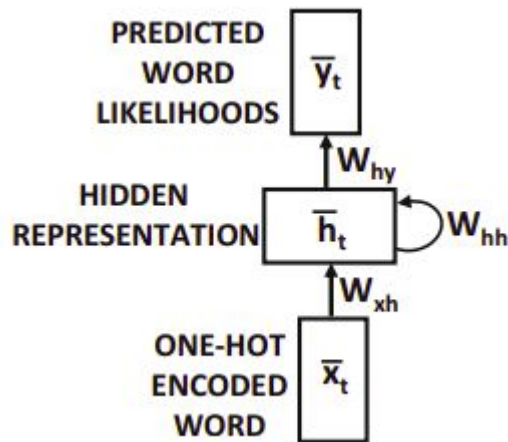
- **Exemplo série temporal:**
 - vetor x_t contém os valores das d variáveis no instante t .
- **Exemplo texto:**
 - vetor x_t é a **one-hot encoding** da palavra no instante t .
 - Tamanho do vetor = tamanho do vocabulário.
 - Ex.: palavra “dog” $\rightarrow [0, 0, 1, 0, \dots, 0]$.

One-hot encoding

		cat	mat	on	sat	the
the =>		0	0	0	0	1
cat =>		1	0	0	0	0
sat =>		0	0	0	1	0
...						
	...					

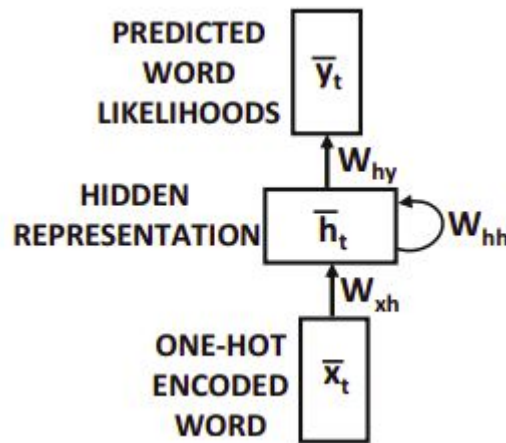
Dependência Sequencial

- Palavras/sinais **dependem do contexto anterior**.
- Uma rede **feedforward tradicional** não modela essa dependência.
- RNN resolve isso:
 - Cada entrada x_t interage com o **estado oculto** h_{t-1} .
 - Assim, a rede “lembra” informações passadas.



Arquitetura básica das RNN

- Em cada instante t :
 - Entrada: x_t
 - Estado oculto: $h_t = f(h_{t-1}, x_t)$
 - Saída: $y_t = g(h_t)$
- **Funções f e g são as mesmas em todos os instantes** (compartilhamento de parâmetros).



Saídas Possíveis

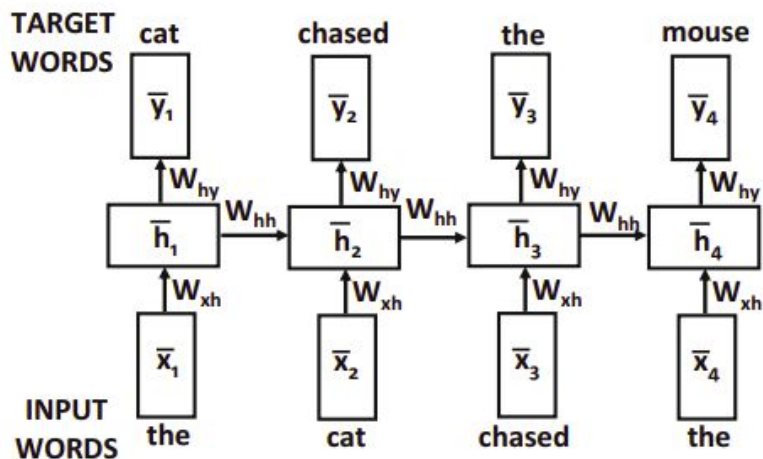
- **Uma saída por passo de tempo:** previsão do próximo valor x_{t+1} .
- **Saída final única:**
 - Ex.: classificar sentimento de uma sentença (positivo/negativo).
- **Modelagem de linguagem:** prever a **próxima palavra** em uma sequência.

Hipótese de Estacionariedade

- Suposição implícita: propriedades estatísticas da sequência não mudam muito ao longo do tempo.
- Ajuda como forma de **regularização**.
- Embora não seja totalmente verdadeira, funciona bem na prática.

0 Looping interno das RNNs

- A principal característica é o **self-loop** no estado oculto
- Após cada entrada **xt**, o **estado oculto ht** é atualizado
- Para facilitar a compreensão, o loop pode ser **desenrolado no tempo** → rede em camadas temporais
- Essa representação é equivalente, mas lembra mais uma rede feedforward



(b) Time-layered representation of (a)

Rede desenrolada no tempo

- Cada **timestamp t** tem seu próprio nó de estado oculto
- Entradas x_t afetam **camadas intermediárias**
- Matrizes de pesos são **compartilhadas** entre os tempos
- Essa **compartilha de parâmetros** permite aprendizado de padrões sequenciais

Backpropagation Through Time (BPTT)

- Treinamento leva em conta a **dependência temporal**
- Gradientes são propagados ao longo dos **passos de tempo**
- Essa técnica é chamada **BPTT**
- Permite treinar funções que dependem de **sequências variáveis**

Funções sobre sequências

- Estado inicial: h_0 fixo (constante)
- Exemplo:
 - $h_1 = f(h_0, x_1)$
 - $h_2 = f(f(h_0, x_1), x_2)$
- Saídas y_t herdam a dependência de todos os $x_1, x_2, \dots, x_t$
- Geral: $y_t = F_t(x_1, x_2, \dots, x_t)$
- Muito útil para **entradas de tamanho variável**, como frases

Propriedades Teóricas das RNNs

- RNNs são **Turing-completas**
 - Podem simular qualquer algoritmo (teoricamente)
- Na prática:
 - Requerem muito **mais dados**
 - Estados ocultos grandes para sequências longas
 - Problemas de **gradiente desaparecendo e explodindo**

Limitações e Variantes

- Problemas práticos:
 - Generalização ruim para sequências longas
 - Dificuldade em escolher hiperparâmetros
- Variantes avançadas:
 - **LSTM (Long Short-Term Memory)**
 - **GRU (Gated Recurrent Unit)**
 - **Neural Turing Machines**

Exemplo

- Código: 1_rnn.py:

Aprendizado Competitivo

Motivação

- A maioria dos métodos vistos até agora → corrigem erros via **backpropagation**.
- Competitive learning é diferente:
 - Não busca mapear entradas → saídas.
 - Objetivo: **neurônios competem** para representar subconjuntos dos dados.
 - Aprendizado **não supervisionado**.

Ideia Central

- Cada neurônio possui um **vetor de pesos** (prototipo).
- Quando chega uma entrada **X**:
 - Mede-se a similaridade entre **X** e cada vetor de pesos.
 - O neurônio **mais parecido** (mais próximo em distância Euclidiana) é o **vencedor**.
- O neurônio vencedor **move seus pesos** em direção ao input.

Winner Take All

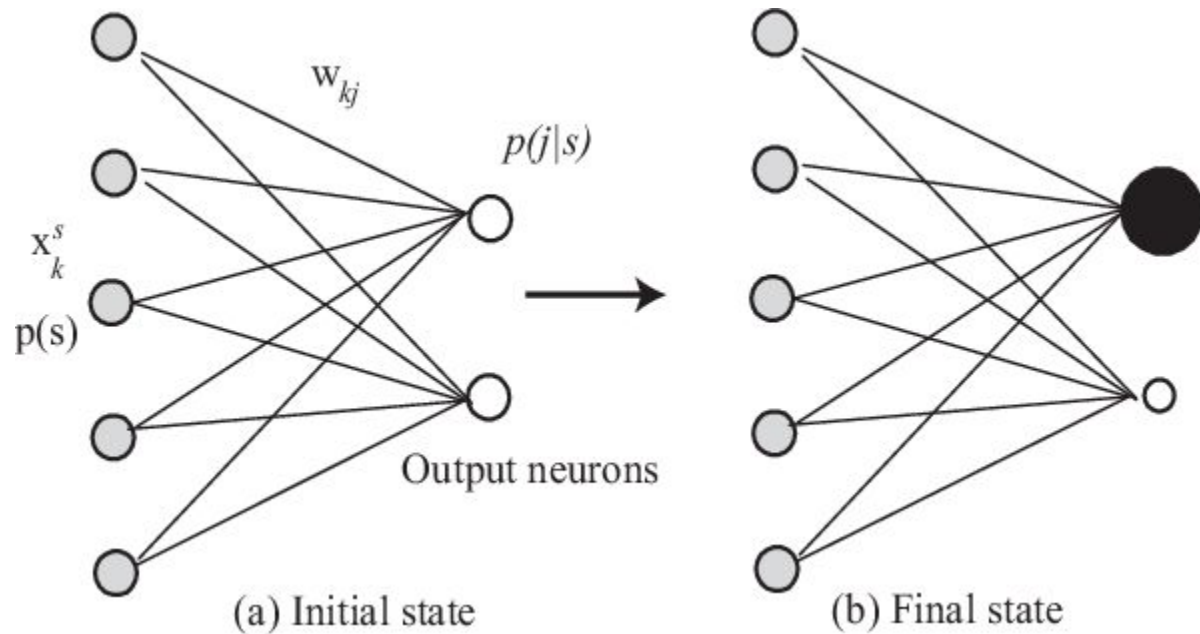
- Estratégia **Winner-Take-All (WTA)**:
 - Só o neurônio **vencedor** é atualizado.
 - Os outros permanecem iguais.
- Variantes:
 - **Neighborhood update** → vizinhos do vencedor também se movem.
 - **Inibição lateral** → neurônios competem ativamente.

Dado:

- Vetor de entrada: $X \in \mathbb{R}^d$
- Pesos: $W_i \in \mathbb{R}^d$ (um para cada neurônio i)
- Taxa de aprendizado: α

Passos:

1. Calcular distância Euclidiana:
 $\|W_i - X\|$ para todo i .
2. Determinar neurônio vencedor p .
3. Atualizar pesos:
 $W_p \leftarrow W_p + \alpha(X - W_p)$



Relação com o K-means

Competitive learning é parecido com **k-means clustering**:

- Ambos associam pontos a **protótipos/centroides**.
- Ambos movem o protótipo em direção ao dado.

Diferença:

- k-means → atualização ocorre **após atribuição de vários pontos**.
- Competitive learning → atualização é **online, ponto a ponto**

- **Clustering não supervisionado**
- **Redução de dimensionalidade**
- **Representações auto-organizadas**
- Exemplos práticos:
 - Mapa de Kohonen (Self-Organizing Maps)
 - Análise exploratória de dados

Vector Quantization

Introdução ao Vector Quantization

- Vector Quantization (VQ) é uma **aplicação simples de aprendizado competitivo**
- Objetivo: representar um conjunto de pontos de alta dimensão com um número reduzido de protótipos (codebook)
- Forte relação com:
 - **k-means clustering**
 - **Compressão de dados**
 - **Redução de dimensionalidade**

Mecanismo de VQ

- Cada neurônio tem um vetor de pesos W_i (prototipo)
- Cada neurônio também possui uma **sensibilidade** $s_i \geq 0$
- Passos principais:
 1. Calcular distância modificada:
$$D_i = \|X - W_i\| - s_i$$
 2. O **vencedor** é o neurônio com menor D_i
 3. Atualização:
 - Para o vencedor: $s_i \leftarrow 0$, e W_i se aproxima de X
 - Para não-vencedores: $s_i \leftarrow s_i + \gamma$

Papel da Sensibilidade

- **Problema:** competitive learning puro pode gerar clusters muito desbalanceados (em regiões densas todos os pontos vão para poucos protótipos).
- **Sensibilidade s_i** corrige isso:
 - Força uma distribuição mais uniforme entre clusters
 - Evita que regiões densas monopolizem os protótipos
- **Parâmetro γ** controla o quanto os não-vencedores acumulam sensibilidade

- Codebook = conjunto de protótipos $W_1, \dots, W_m$
- Cada ponto é armazenado apenas como o índice do protótipo mais próximo
- Vantagem:
 - Espaço reduzido de $n \cdot d$ para $m \cdot d + n \cdot \log_2(m)$
- **Exemplo numérico:**
 - Dataset: 10 bilhões pontos, 100 dimensões, 4 bytes por dimensão $\rightarrow \sim 4$ TB
 - VQ com $m = 10^6$:
 - Codebook < 0.5 GB
 - Dados ~ 3 GB
 - Total < 3.5 GB (vs 4 TB originais!)

Compressão com perda (lossy)

- Aproxima cada ponto pelo vetor W_i mais próximo
- Erro de reconstrução:
 $// X - W_i //$

Áreas de aplicação

- Compressão de imagens e áudio
- Codificação de vídeo (MPEG, MP3 usam VQ internamente)
- Redução de dimensionalidade
- Pré-processamento para algoritmos supervisionados

- Ver código `3_vector_quantization.py`
- Neste código:

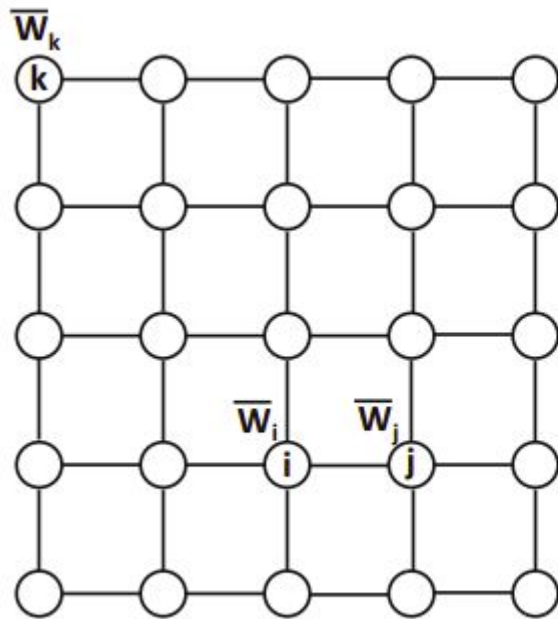
Mapa Auto-Organizável de Kohonen

Cap. 10.5

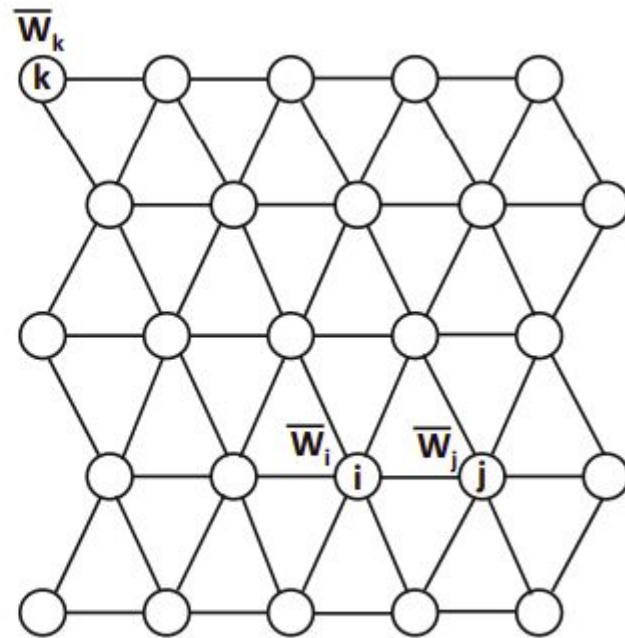
- O **Mapa Auto-Organizável de Kohonen (SOM)**
- Variação do **paradigma de aprendizado competitivo**
- Impõe uma **estrutura em grade (lattice)** sobre os neurônios:
 - 1D: formato de corda (string-like)
 - 2D: formato de grade (lattice-like)

Estrutura em grade

- Cada neurônio tem um vetor de pesos W_i
- Os neurônios são organizados em uma grade **2D** (ex: $5 \times 5 = 25$ neurônios)
- **Exemplos:**
 - Grade retangular
 - Grade hexagonal
- A forma da grade afeta a forma das regiões de clusters.



(a) Rectangular



(b) Hexagonal

- **Distância no espaço dos dados (vetores):**

$$Dist(W_i, W_j) = ||W_i - W_j||$$

- **Distância na grade (lattice):**

$$LDist(i, j)$$

- Depende apenas das posições i, j na grade
- Exemplo: vizinhos diretos $\rightarrow 1$ unidade

Ideia Central

- Vizinhos na **grade** devem ter vetores de peso semelhantes
- A **estrutura da grade** atua como um **regularizador**
- Permite projetar dados em **2D para visualização**

Algoritmo de Treinamento

1. Amostra X do conjunto de treino
2. Acha o **neurônio vencedor** ppp:

$$p = \arg \min_i ||X - W_i||$$

3. Atualiza os pesos:

$$W_i \leftarrow W_i + \alpha \cdot Damp(i, p) \cdot (X - W_i)$$

Função de Amortecimento

- Define quanto os vizinhos do vencedor são atualizados
- Geralmente uma gaussiana:

$$Damp(i, j) = \exp \left(-\frac{LDist(i, j)^2}{2\sigma^2} \right)$$

- **σ pequeno** $\rightarrow$ Winner-Take-All (só o vencedor é atualizado)
- **σ grande** $\rightarrow$ Mais regularização (vizinhos também aprendem)

Convergência

- Iterar até que pesos se estabilizem
- **Neurônios vizinhos** ficam com vetores semelhantes
- Resultado: mapa 2D que preserva **relações topológicas** dos dados originais

Benefício do SOM

- Reduz a **dimensionalidade** para 2D
- Facilita **visualização de clusters**
- Útil para **descoberta de padrões** em dados de alta dimensão

Exercício 1

- Use o código `1_rnn.py` para gerar os vetores de características (embeddings) de cada amostra do conjunto de treinamento. Salve-os na pasta `embeddings/`
- Use o código `competitive_learning_cluster.py` para ler esses embeddings e agrupá-los em 3 grupos (usando 3 protótipos).
- Pergunta: Depois que eu achar os 3 protótipos, como eu faço a inferência de uma nova amostra? Ou seja, como eu encontro o grupo de uma nova amostra?

Exercício 2

- No Classroom. Entrega até semana que vem.